



第 23 课 - 函数高阶

● 闭包

✧ 闭包, 又称闭包函数或者闭合函数。其实和前面讲的嵌套函数类似, 不同之处在于:

■ 闭包中外函数返回的不是一个具体的值, 而是一个函数。

✧ 一般情况下, 返回的函数会赋值给一个变量, 这个变量可以在后面被继续执行调用。

```
01. def fun(y):  
02.     def indef(x):  
03.         return x ** y  
04.     return indef # 返回值是 indef 函数  
05. a = fun(2) # 计算一个数的平方  
06. b = fun(3) # 计算一个数的立方  
07. print(a(2)) # 计算 2 的平方  
08. print(b(2))
```

✧ 好处: 当需要多次调用该函数时, 如果将那些额外工作的代码放在外部函数, 就可以减少多次调用导致的不必要开销, 提高程序的运行效率。

● lambda 表达式

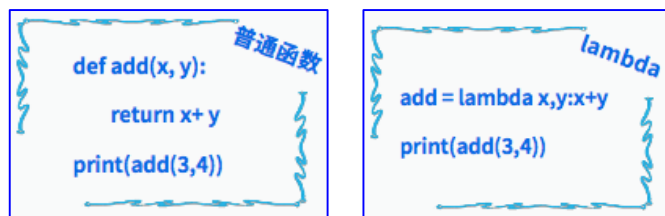
✧ lambda 表达式 -- 用于定义一个简单的函数

✧ lambda 表达式, 又称匿名函数。常用来表示内部仅包含 1 行表达式的函数。如果一个函数的函数体仅有 1 行表达式, 则该函数就可以用 lambda 表达式来代替。

✧ 语法:

■ **name = lambda [list]: 表达式**

- ◆ 定义 lambda 表达式，必须使用 lambda 关键字。
- ◆ [list] 作为可选参数，等同于定义函数是指定的参数列表；
- ◆ name 为该表达式的名称。

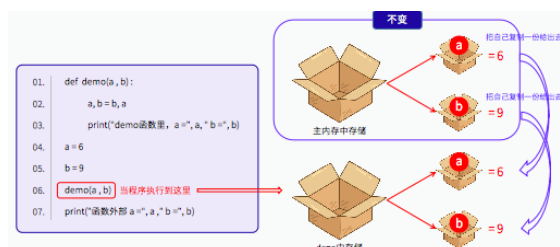


◇ 相比函数，lambda 表达式具有以下 2 个优势：

- 对于单行函数，使用 lambda 表达式可以省去定义函数的过程，让代码更加简洁；
- 对于不需要多次复用的函数，使用 lambda 表达式可以在用完之后立即释放，提高程序执行的性能。

● 值传递的传递机制

- ◇ 值传递，本质是将实参值复制一份，将其副本传给形参。
- ◇ 采用值传递方式的函数中，无论其内部对参数值进行如何修改，都不会影响函数外部的实参。



◇ 值传递的实质：

- 当系统开始执行函数时，系统对形参执行初始化，就是把实参变量的值赋给函数的形参变量，在函数中操作的并不是实际的实参变量。

- 引用传递的传递机制

- ✧ 实参是可变对象（列表、字典） -- 引用传递。

- ✧ 修改函数内的值，会导致函数外的值随之改变。

